

Job-Pilot — Update 0.1.0 (Detailed, Categorized)

Date: 2026-06-18

Author: development pass (local-model migration + bug-fix sweep + auto-fill engine)

This document organizes every change made in this update **by category** —

AI, Backend, Automation (auto-apply), Database, UI/Frontend, Configuration, Data flow, and Testing — followed by known limitations / backlog.

Companion file: `update_0.1.0.md` (the chronological narrative). This file is the categorized reference.

0. Executive summary

Three workstreams shipped in this update:

1. **Local-model migration** — all AI tasks moved from cloud (Anthropic Claude + Google Gemini) to a single local open-source model (Ollama / `qwen2.5:3b`) for **testing the workflow and data lineage**, at **\$0** and fully offline. Cloud config is preserved (commented) for one-step revert.

1. **Bug-fix sweep** — onboarding, dashboard, applications, settings, navigation, documents, and preference-aware job matching.

1. **Auto-apply field-resolution engine** — a vendor-agnostic browser form-filler that fills **every** field type by meaning (text, select, radio, checkbox, react-select combobox), handles cross-origin ATS iframes, conditional fields, demographics, and verifies coverage. Validated end-to-end on the real Stripe→Greenhouse application form: **21/21 fields filled, only Submit remains**.

1. AI / Models

Task	Before	After
Resume tailoring	Anthropic Claude (Sonnet)	local <code>qwen2.5:3b</code>
Cover letter	Gemini Pro	local <code>qwen2.5:3b</code>

Task	Before	After
Resume parse	Gemini	local <code>qwen2.5:3b</code>
Job parse	Gemini	local <code>qwen2.5:3b</code>
Match score	Gemini	local <code>qwen2.5:3b</code>
Cold email	Gemini	local <code>qwen2.5:3b</code>
Question answering	Gemini	local <code>qwen2.5:3b</code>
Embeddings	<code>text-embedding-004</code>	local <code>nomic-embed-text</code> (768-dim)

Files

- `server/services/ai/model-config.ts` — `tailorResume` flipped from Anthropic to

the local OpenAI-compatible provider; original Claude line kept as a comment.

- `server/services/ai/prompts.ts` — **question-answering prompt rewritten** to feed

the model BOTH the full **job description (what the employer wants)** and the candidate's **résumé text (what they actually did)**, with explicit instructions

to connect the two using only real facts (`buildQuestionPrompt` gained a

`resumeText` arg).

Why `qwen2.5:3b` (not a "thinking" model): `qwen3:*` route their `<think>`

output away from `content` on the OpenAI-compatible path, so with the app's small

per-task `max_tokens` the response came back **empty**. `qwen2.5:3b` returns clean

JSON immediately.

Quality note: the grounding/logic is correct (JD+résumé analysis, fact-only

constraints, retry-on-refusal), but prose polish/consistency on nuanced prompts is

bounded by the local 3B test model — the production model will sharpen it.

2. Backend (app-tier services & routes)

2.1 Question answering (`server/services/application/qa-generator.ts`)

- **Threshold made env-configurable** (`config.qa.confidenceThreshold`); AI gate

honors `config.qa.answerAll` (testing) so every non-sensitive open question is

drafted instead of only `why-this-company` -style ones.

- **Employment-history yes/no answered from facts** — "Have you ever been employed

by Stripe / a Stripe affiliate?" derives ****Yes** only if the named company is among

the candidate's actual employers, else **No**** — never a false affirmative.

- **JD + résumé grounding** for open-ended questions (passes `resumeText`).
- **Retry-on-refusal** — a one-shot firmer retry pushes weak local models past a

spurious `NEEDS_USER_ACTION` when résumé grounding exists.

2.2 Candidate profile (`server/services/profile/candidate-profile.ts`)

- `profileSummary()` enriched with the candidate **summary + up to 3 recent roles**

(concrete material for "describe your experience" answers).

- `workAuthorization` **derived** from the structured `requiresSponsorship` +

`visaStatus` pair (the free-text field was deduplicated on the editor).

- `websiteUrl` resolves canonical `portfolioUrl` first (personal-website dedup).

2.3 Preference-aware ingestion (`server/services/ingestion/ingestion-orchestrator.ts`)

- Loads the user's `targetRoles` and applies a deterministic **role filter** —

a scraped title must contain all distinctive tokens of a target role

(engineer/engineering stemmed). Off-target roles are never ingested (fixed the

"every Stripe role at fake ~85%" problem).

2.4 Config (`server/lib/config.ts`)

- Added `numFloat()` helper.
- `qa: { confidenceThreshold, answerAll }.`
- `automation.headed` (show the browser window in local testing).

2.5 Routes (`server/routes/applications.ts`)

- `GET /api/applications/documents` (registered BEFORE `/:id`) — feeds the new

Documents page.

- `POST /api/applications/:id/submit` drives the auto-fill engine with JD + profile

context and persists the outcome + `filledFields` to `ApplicationEvent` .

3. Automation — auto-apply field-resolution engine

(`server/services/automation/form-filler.ts`)

The largest change. A vendor-agnostic engine that fills a live application form by

meaning, not brittle selectors.

3.1 Scrape every control

`scrapeControls()` tags and extracts **all** interactive controls — text, textarea, native `<select>`, radio groups, checkboxes, and custom **comboboxes (react-select)** — with their real human label, options, required flag (via the `*` marker), and current filled-state. Skips noise: react-select's phantom `requiredInput`, the intl-tel phone-country search box, and the reCAPTCHA textarea.

3.2 Resolve each field by meaning (strategy ladder)

Field category	Strategy
Personal / professional / education / links	deterministic label→profile (no LLM)
Same question reworded	semantic embedding match on stored answers
Immigration / work-authorization	grounded ONLY from profile facts (never invented)
Open-ended ("Why this company?")	AI using JD + résumé
Employment history ("Employed by X?")	derived from the candidate's employers
Demographic / EEO	the candidate's OWN provided self-ID, else "Decline"

3.3 Apply per widget type

fill / `selectOption` / check radio / tick consent checkbox / drive react-select (click the **control container** to open → read options → type-to-filter for long lists with empty-filter restore → pick). Cross-vendor **representation mapping** via an option-constrained LLM pick (e.g. "United States" → **"US"**, verbose yes/no → the right option).

3.4 Voluntary EEO uses the user's own answer

`eeoValue()` returns the candidate's provided Gender / Race / Veteran / Disability value (mapped to the vendor's exact options); only **blank** fields fall back to **"Decline to self-identify."** The Hispanic/Latino yes-no is derived from race ("Asian" → "No"). It never invents or asserts a demographic.

3.5 Conditional fields

STEP 3 runs as a **re-scan loop** (max 4 passes): after each fill pass it re-scrapes and fills anything newly **revealed** by an earlier answer — e.g. answering "Are you Hispanic/Latino?" unhides "Please identify your race", which then fills (Asian).

3.6 Coverage verification

`computeCoverage()` re-scrapes and reports required-filled vs blanks → an honest

"only Submit remains" or the exact fields still needing input. Submission stays

gated by `AUTO_SUBMIT` (default off → fill + stop for user review).

3.7 Hard problems solved on the real form

- **Collapsed ATS child-iframe** — Stripe renders the Greenhouse iframe collapsed (fields exist in the DOM, but react-select can't open). Fix: `**promote the embed to a top-level navigation**`.

- **react-select drops element tags on selection** → drive comboboxes by their **stable DOM id**, not the transient `data-jp` tag.

- **Filled-state detection** — read at `.select__control` / `.select-shell`, not the inner `input-container` (which sits beside the value node).

- `__name is not defined` in `page.evaluate` — tsx/esbuild `keepNames` shim.

4. Database (PostgreSQL / Prisma)

- Local `jobpilot` DB **reset and migrated** to the collaborator's merged schema (all 5 migrations: `init`, `email_verification`, `postgres_artifacts`, `indexes_and_audit`, `plan_daily_cap`).

- **Prisma client regenerated** (v7 → `node_modules/@prisma/client`); fixed stale `Unknown argument` errors (`Plan.applicationsPerDay` , `UserProfile.consentToDataProcessing`).

- EEO columns used (already present): ``UserProfile.gender / raceEthnicity / veteranStatus / disabilityStatus`` — now populated via the UI and consumed by the fill engine.

- No schema migration added in this update (app-tier + UI only).

5. UI / Frontend (React + Vite)

#	Area	Change	File
1	Onboarding	Fixed input focus-loss (component hoisted to module scope)	<code>src/pages/onboarding/steps/step-application-details.tsx</code>
2	Applications	Fixed "Something went wrong" crash — global <code>TooltipProvider</code>	<code>src/app/providers.tsx</code>
3	Settings	Approval mode now loads + persists (was a stub)	<code>src/pages/settings/settings.tsx</code>
4	Dashboard	Stat tiles clickable → deep-link to Jobs / Applications / Review	<code>src/pages/dashboard/dashboard.tsx</code>
5	Applications	Reads <code>?status=</code> from URL	<code>src/pages/applications/applications.tsx</code>
6	Documents	New Documents view (cover letters / cold emails / tailored resumes)	<code>src/pages/resume/resume.tsx</code> , <code>src/services/api/applications.ts</code>
7	Navigation	Retired duplicate "Candidates"; "Jobs" → "Jobs Found", "Resume" → "Documents"	<code>src/components/layout/sidebar.tsx</code>
8	Onboarding	Removed repetitive questions (sponsorship/visa/personal-website dupes)	<code>src/pages/onboarding/steps/step-application-details.tsx</code>
9	EEO dropdowns	Voluntary self-ID is now dropdowns (standard EEOC option lists) in BOTH the profile editor AND the signup wizard; copy clarifies values are used to auto-fill but never submitted without review	<code>src/pages/profile/profile-editor.tsx</code> , <code>src/pages/onboarding/steps/step-application-details.tsx</code> , new <code>src/lib/eeo-options.ts</code>

6. Configuration / Environment (`.env` , `gitignored`)

- **Ollama** provider: `AI_COMPAT_BASE_URL=http://localhost:11434/v1` ,

`AI_MODEL_FLASH/PRO=qwen2.5:3b` , `AI_EMBED_MODEL=nomic-embed-text` . Gemini block

preserved as comments for revert.

- **Automation (testing):** `AUTOMATION_MODE=auto` , `AUTO_SUBMIT=false` ,

`PLAYWRIGHT_HEADED=true` — drives a visible browser to FILL and ****stop** before

Submit**.

- **Q&A (testing):** `QA_CONFIDENCE_THRESHOLD=0.4` , `QA_ANSWER_ALL=true` — draft &

fill every open question; restore prod defaults (`0.75` / `false`) later.

7. Data flow (end to end)

1. **Ingestion** → scrape boards → **role-filter against** `targetRoles` → store only on-target jobs.
 1. **Match / tailor / generate** → local model scores, tailors résumé, writes cover letter / cold email; embeddings power semantic Q&A reuse.
 1. **Auto-apply** → `POST /:id/submit` → launch Playwright → promote ATS iframe to top-level → scrape all controls → resolve each by meaning (profile / semantic / JD+résumé / EEO) → apply per widget → **re-scan loop** for conditional fields → **verify coverage** → stop before Submit (AUTO_SUBMIT off).
 1. **Review** → user inspects the prepared/filled application and submits.
-

8. Testing & verification

- End-to-end against the real **Stripe** → **Greenhouse** form ("Fullstack Engineer, Privy), `headless and headed`, `AUTO_SUBMIT=false`.
 - Result, stable across repeated runs: **All 16 required fields filled, 21/21 total — "only Submit remains."**
 - Verified values: identity/contact/employment, Country (US↔United States mapping), Location, work-auth, sponsorship, open-ended "Why Privy?" (JD+résumé), "Employed by Stripe?" → **No**, and demographics from the user's own profile (Gender=Male, Hispanic/Latino=No, Veteran="not a protected veteran", Disability="No...", Race=Asian via the conditional field).
 - Dev helper retained: `scripts/test-autofill.ts` (mints a JWT, calls `/submit` , prints the coverage report). No secrets committed (`.env` gitignored).
-

9. Known limitations / backlog

- **Cross-vendor verification** — engine validated on Greenhouse; verify Lever / Ashby / Workable (combobox markup + file-upload widget).
- **TESTING crutches active** (`config.qa.answerAll`): a required option the model can't resolve gets a **negative/safe** option (never a blind "Yes"); restore prod

Q&A thresholds before launch.

- **Answer prose quality** is bounded by the local 3B; the prod model fixes it.
- **Match-score quality** — weak local model rubber-stamps; add a deterministic

role/skill-fit component or a stronger model.

- **More scrapable companies** — Google/Amazon use Workday/custom ATS; add adapters.
- **Production model roadmap** — eval set → distill → QLoRA → serve on vLLM/TGI on a

real GPU (swap `AI_COMPAT_BASE_URL` , zero app-code change).